# Implementation of Boolean Logic Operation in Embedded Flash Memory for In-Memory Computing Applications

Jinhyeok Kim, Sung Jin Bang, Min Sung Kim, Minsuk Koo, *Member, IEEE*, and Yoon Kim, *Member, IEEE*

***Abstract*— This article presents, for the first time, a sequential logic-in-memory technique using embedded flash (eFlash) memory to address the integration challenges of previous methods that relied on memristor devices. In this work, Boolean primitives are executed and retained as a nonvolatile cell state inside the eFlash array, while cascaded operations are performed through a system-level sequential execution. The incompatibility of memristor devices with standard CMOS logic processes has limited their practical integration. The proposed eFlash-based approach offers full compatibility with CMOS circuits, resulting in improved operating speed and reduced energy consumption. A test chip was designed and fabricated using a 180-nm low-power CMOS process, successfully demonstrating arbitrary Boolean logic operations on a fabricated embedded NOR flash memory test chip.**

***Index Terms*— Boolean logic operation, embedded flash (eFlash) memory, embedded nor flash memory, in-memory computing.**

## I. Introduction

CURRENT computing systems are predominantly based on the von Neumann architecture, characterized by the physical separation of the processing and memory units. While this division of labor has historically been efficient and instrumental in advancing electronic systems, the advent of the big data era has highlighted significant limitations in terms of speed and energy efficiency. Specifically, the "memory wall" phenomenon, resulting from the speed disparity between the processing and memory units, and the considerable energy consumption due to data movement between these units, are major concerns [1]. To address these issues, researchers have proposed in-memory computing, wherein the operations of the arithmetic logic unit are performed within the memory unit [2], [3], [4], [5], effectively eliminating inefficient data movement.

Stateful logic based on in-memory computing is a technique wherein data stored in nonvolatile memory devices is utilized as logical states to perform computations, and the results are subsequently recorded back into the memory devices as logical states [6], [7], [8]. In earlier method, all 16 Boolean logic functions were implemented through iterative applications of IMP and FALSE operations [9]. To eliminate the need for iterative steps, sequential logic [5], [10], [11], [12], MAGIC logic [13], [14], [15], and FELIX [16] are proposed. Boolean logic functions based on stateful logic are implemented using various nonvolatile memory devices such as phase change memory (PRAM) [17], resistive random access memory (RRAM) [5], [18], [19], magnetic random access memory (MRAM) [20], ferroelectric field effect transistor (FeFET) [21], and charge trap flash (CTF) memory [22], [23].

While these emerging memory devices demonstrate promising cell characteristics, they are not fully compatible with standard CMOS logic processes for front-end-of-line (FEOL) integration, making it challenging to integrate them with CMOS driving circuits on the same die plane. Although some foundries have begun offering embedded NVM technologies—such as RRAM and MRAM—via back-end-of-line (BEOL) integration or 3-D monolithic stacking approaches, these methods are fundamentally different from FEOL integration. In particular, BEOL-based embedded flash (eFlash) memories face difficulties in process optimization due to thermal budget conflicts with the underlying CMOS logic layers. Furthermore, in BEOL structures, signal transfer between memory and logic circuits occurs through vertical interconnects, such as through-silicon vias (TSVs) or

Received 22 October 2025; revised 23 December 2025; accepted 23 January 2026. Date of publication 12 February 2026; date of current version 26 February 2026. This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by Korean Government (MSIT) under Grant 2021-0-01764, and in part by the National Research and Development Program through the National Research Foundation of Korea (NRF) funded by Ministry of Science and Information and Communication Technology (ICT) under Grant 2022M3I7A1078544 and Grant RS-2021-NR057239. The work of Yoon Kim was supported by the 2024 Research Fund of the University of Seoul. The review of this article was arranged by Editor S. Raghunathan. *(Corresponding authors: Yoon Kim; Minsuk Koo.)*

Jinhyeok Kim and Min Sung Kim are with the School of Electrical and Computer Engineering, University of Seoul, Seoul 02504, South Korea.

Sung Jin Bang is with Samsung Electronics Company Ltd., Hwasung 445-701, South Korea.

Minsuk Koo is with the School of Advanced Fusion Studies and AI Semiconductor, University of Seoul, Seoul 02504, South Korea, and also with IM Electronics Company Ltd., Seoul 02504, South Korea (e-mail: koo@uos.ac.kr)

Yoon Kim is with the School of Electrical and Computer Engineering, University of Seoul, Seoul 02504, South Korea, and also with IM Electronics Company Ltd., Seoul 02504, South Korea (e-mail: yoonkim82@uos.ac.kr).

Digital Object Identifier 10.1109/TED.2026.3659834

1557-9646 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:34:54 UTC from IEEE Xplore. Restrictions apply.

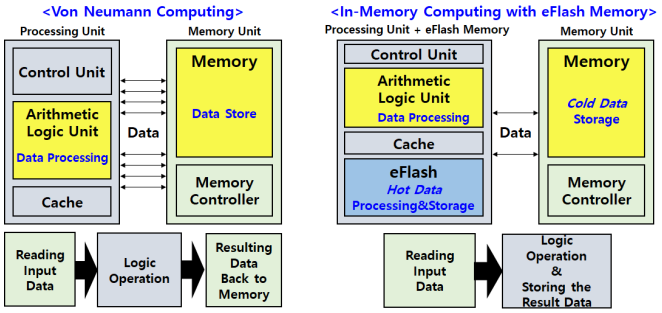



Fig. 1. Comparison between the conventional von Neumann computing architecture and the proposed in-memory computing architecture with eFlash memory.

monolithic interlayer vias (MIVs), which inherently introduce additional access latency compared to configurations where memory and logic are located on the same plane.

In this study, for the first time, we propose an eFlash memory-based sequential logic-in-memory technique. While stateful logic ideally retains both inputs and outputs as memory states, our approach is distinct in that only the result is stored as a state, necessitating an additional readout and voltage conversion step for the inputs. This allows the integration of nonvolatile memory devices and the CMOS driving circuits required for implementing Boolean logic functions with minimal physical distance, providing the advantages of increased operating speed and reduced energy consumption. Importantly, the proposed technique is not designed to replace all logic operations within a computing system, nor is it intended to serve as a high-density data storage solution like conventional NAND flash memory. Instead, it is selectively applied to logic operations involving *hot data*—frequently accessed or immediately processed data—that require direct storage in nonvolatile memory (Fig. 1). Here, *hot data* refers to small-volume data that is frequently updated yet must be persisted immediately (e.g., IoT sensor logging records, edge/on-device learning weight updates, and system configuration/state updates) [24]. In such cases, our eFlash primitives are well suited to process and retain updates in-place, reducing the need to route every update through the CPU and external transfer path. In conventional systems, logic results are typically transferred back to memory through separate write operations, which introduce significant latency and energy overhead. By integrating logic and storage directly within the memory array—only where such functionality is needed—the proposed method eliminates unnecessary data movement and energy cost while maintaining compatibility with existing CMOS logic circuits.

Furthermore, this approach is particularly well suited for application-specific scenarios such as on-device AI, where a limited set of logic operations are frequently executed and the results must be retained in-place for energy-efficient processing. In such cases, high storage density is not a primary requirement; rather, low-latency, nonvolatile logic-in-memory execution becomes the key design goal. This selective application makes the approach suitable for practical deployment as an auxiliary logic layer in CPU chips, complementing conventional logic and memory systems rather than replacing them.

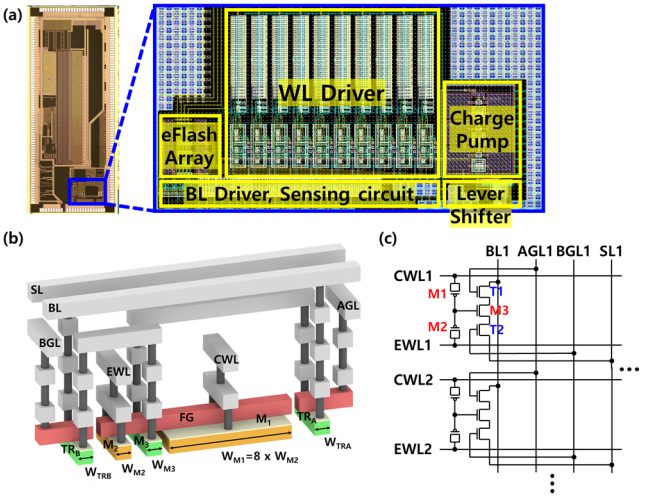



Fig. 2. (a) Microscopic image and layout of fabricated chip, (b) 5T single-poly eFlash cell 3-D structure, and (c) schematic of NOR eFlash memory array.

Here, we experimentally demonstrate arbitrary Boolean logic using a fabricated embedded NOR flash memory chip.

## II. Logic-Compatible Embedded NOR Flash Memory Array

The nonvolatility of eFlash allows it to retain data without power, making it indispensable for applications requiring persistent storage, such as firmware, configuration data, and user settings. The ease of integration of eFlash into CMOS-based system-on-chip designs not only streamlines system architecture by reducing the need for external memory components but also enhances performance and security through features such as secure boot and encryption. The structure of eFlash devices can be broadly categorized into traditional dual-poly [25], split-gate [16], [27], and single-poly [28], [29], [30], [31] eFlash technologies. Among these, single-poly eFlash can be embedded without additional process overhead, as it can be implemented using discrete *I*/*O* transistors readily available in standard logic process [28]. A test chip was fabricated using a 180-nm low-power CMOS process, incorporating eFlash array along with the peripheral circuits such as wordline (WL) driver, bitline (BL) driver, sensing circuit, charge pump, and level shifter as shown in Fig. 2(a). We utilized the 5T single-poly eFlash technology [31] as illustrated in Fig. 2(b). The three MOS capacitors (M1, M2, M3) share a floating gate (FG) and are connected to two MOSFETs (T1, T2) at both ends. For rowwise parallel stateful logic operations, we modified the connection direction of control wordline (CWL) and erase wordline (EWL) in the previous array structure as depicted in Fig. 2(c).

In the eFlash cell, the n-well of the coupling device M1 functions like the gate of a conventional flash memory cell. To ensure that the CWL can determine the electrical potential of the FG, a high coupling ratio between the n-well of M1 and the FG must be achieved. This can be readily accomplished by increasing the size of the M1 device. If the width of M1 is too small, the program operation voltage tends to increase, whereas an excessively large width leads to an increase in the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:34:54 UTC from IEEE Xplore. Restrictions apply.

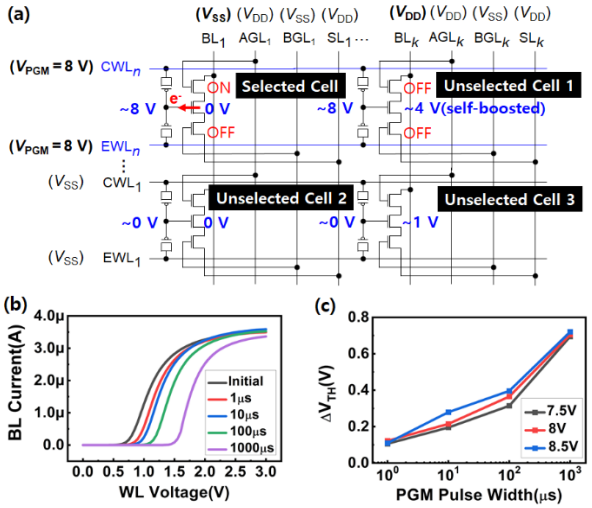



Fig. 3. (a) Operating voltages of selected/unselected cells in PGM operation, (b) *I*–*V* curves as PGM, and (c) $V_{TH}$ variance for different PGM pulse widths.

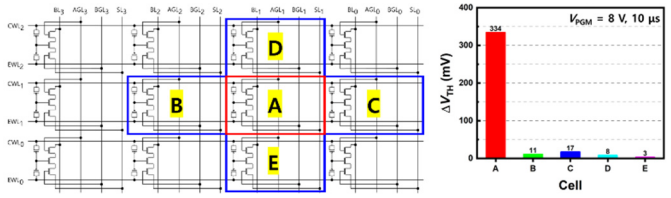



Fig. 4. PGM disturb evaluation for adjacent cells sharing the same WL or BL.

overall device area. To address this tradeoff, the width of M1 was designed to be eight times larger than that of either M2 or M3, reflecting the optimized design reported in [31], which aims to achieve program operation at voltages below 8 V. For program (PGM) operation, $V_{DD}$ is applied to the A gate line (AGL) and source line (SL) of the selected cell and $V_{SS}$ (0 V) is applied to the B gate line (BGL) and BL, as illustrated in Fig. 3. This maintains T1 in an ON state and T2 in an OFF state; then, a high program pulse (e.g., $V_{PGM}$, 8 V) is applied to both the CWL and EWL of the selected cells. Due to the capacitance between the body and gate of the PMOS devices (M1, M2), capacitive coupling occurs, leading to an increase in the electrical potential of the FG node. At this point, the body of M3 forms an n-channel and is connected to the 0 V of BL1 through the T1 transistor. Consequently, the significant voltage difference between the FG node and the channel initiates Fowler–Nordheim (FN) tunneling, resulting in electron injection into the FG node of the M3 device (State 0). In the case of unselected cell 1, since both T1 and T2 are OFF state, the channel voltage of M3 becomes floating. This floating condition, combined with the high programming voltage ($V_{PGM}$) applied to EWL and CWL, induces self-boosting. As a result, due to the small electric field between the FG node and the channel, FN tunneling is inhibited. For unselected cells 2 and 3, tunneling does not occur as $V_{SS}$ is applied to both EWL and CWL. The programming measurement results of the selected cell are shown in Fig. 3(b) and (c). Furthermore, program disturb characteristics were evaluated by applying a program pulse to

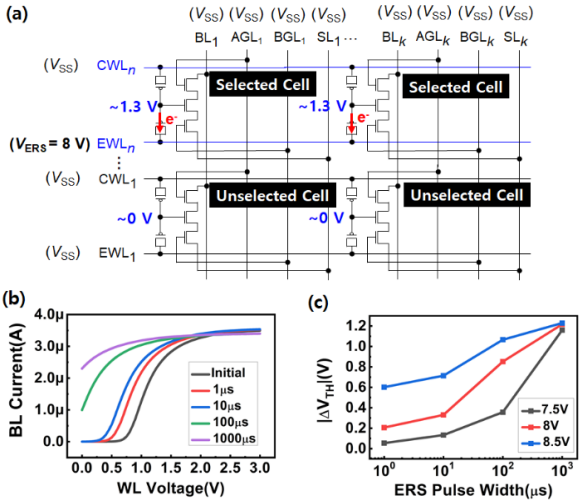



Fig. 5. (a) Operating voltages of selected/unselected cells in ERS operation, (b) *I*–*V* curves as ERS, and (c) $V_{TH}$ variance for different ERS pulse widths.

the target cell while monitoring adjacent cells sharing the same wordline or bitline (BL). As shown in Fig. 4, the threshold voltage shift in neighboring cells was less than 5% of that observed in the programmed cell, indicating that the proposed biasing scheme effectively suppresses program disturbance in unselected cells.

The erase operation is performed on a WL basis. As shown in Fig. 5, when the erase voltage ($V_{ERS}$) is applied to the EWL and $V_{SS}$ (0 V) is applied to the remaining metal lines, the FG node attains a low potential (~1.3 V). At this point, the electrons stored in the FG tunnel out through the gate dielectric of M2, while holes tunnel into the FG.

## III. Boolean Logic Operation in Embedded NOR Flash Memory Array

To implement a Boolean logic function on an embedded NOR flash memory array, it is necessary to define the cell state and operation control signals in correspondence with the logic variables. Operation control signals refer to the voltages applied to BL, AGL, BGL, and SL within the array, where $V_{SS}$ represents State "0" and $V_{DD}$ represents State "1." When $V_{PGM}$ is applied to EWL and CWL, the final cell state result (R) of the corresponding cell is determined by the operation control signals. In our logic concept, the cell state with a high threshold voltage, resulting from programming (PGM), is defined as State "0," while the cell state with a low threshold voltage, resulting from erasing (ERS), is defined as State "1." This relationship is represented by the Karnaugh map shown in Fig. 6. The resulting expression is given as follows:

$$R = \mathrm{BL} \cdot \overline{\mathrm{BGL}} + \overline{\mathrm{AGL}} \cdot \overline{\mathrm{BGL}} + \mathrm{BL} \cdot \mathrm{SL} + \overline{\mathrm{AGL}} \cdot \mathrm{SL}. \quad (1)$$

One important point to emphasize is that in the cases where (BL, AGL, BGL, SL) = (0, 1, 1, 1) or (1, 1, 1, 0), both T1 and T2 turn on simultaneously, causing DC current to flow from BL and SL. This leads to significant power consumption, so these state combinations have been excluded as prohibited states.

Table I presents the combinations of operation control signals required to implement 16 arbitrary Boolean logic

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:34:54 UTC from IEEE Xplore. Restrictions apply.

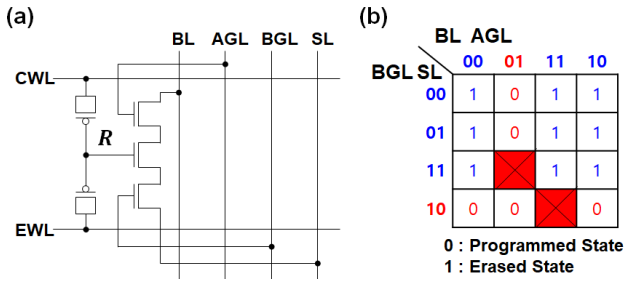



Fig. 6. (a) eFlash NOR memory cell structure and (b) Karnaugh map of BL, AGL, BGL, and SL for Boolean logic operation in NOR eFlash memory array.

TABLE I
OPERATION CONTROL SIGNAL ASSIGNMENT FOR BOOLEAN LOGIC OPERATIONS

| Boolean | BL | AGL | BGL | SL |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 |
| $p$ | $p$ | 1 | 0 | 0 |
| $q$ | $q$ | 1 | 0 | 0 |
| $\bar{p}$ | 0 | $p$ | 0 | 0 |
| $\bar{q}$ | 0 | $q$ | 0 | 0 |
| $p+q$ (OR) | $q$ | $\bar{p}$ | 0 | 0 |
| $p \cdot q$ (AND) | $p$ | $\bar{p}$ | $\bar{q}$ | $q$ |
| $\overline{p+q}$ (NOR) | 0 | $p$ | $q$ | 0 |
| $\overline{p \cdot q}$ (NAND) | $\bar{q}$ | $p$ | 0 | 0 |
| $p+\bar{q}$ (RIMP) | $p$ | $q$ | 0 | 0 |
| $\bar{p}+q$ (IMP) | $q$ | $p$ | 0 | 0 |
| $p \cdot \bar{q}$ (RNIMP) | $p$ | $\bar{p}$ | $q$ | 0 |
| $\bar{p} \cdot q$ (NIMP) | 0 | $p$ | $\bar{q}$ | $q$ |
| $\bar{p} \cdot q + p \cdot \bar{q}$ (XOR) | $\bar{q}$ | $p$ | $\bar{p}$ | $q$ |
| $p \cdot q + \bar{p} \cdot \bar{q}$ (XNOR) | $q$ | $p$ | $q$ | $p$ |

functions. The proposed method is not the only solution; it can also be implemented with different combinations according to (1). Based on the definition above, all 16 of the Boolean logic functions can be processed using the following four sequential steps.

*Step 1 (Initialize):* The target cells are set to State "1" through the ERS operation. This step may be omitted if the cells are already in the erased state.

*Step 2 (Precharge):* The channels of the M3 devices in the target cells are precharged by applying $V_{DD}$ to the CWL, EWL, BL, and AGL.

*Step 3 (Voltage Setting):* The appropriate operation control signals are applied to BL, AGL, BGL, and SL in accordance with Table I to execute the desired Boolean logic operation.

*Step 4 (Program):* A program voltage ($V_{PGM}$) is applied to the CWL and EWL of the target cells to execute the PGM operation. At this stage, the result of the Boolean logic operation is stored in situ in the target cell.

To implement all 16 Boolean logic functions using conventional CMOS logic would require approximately 78 transistors.

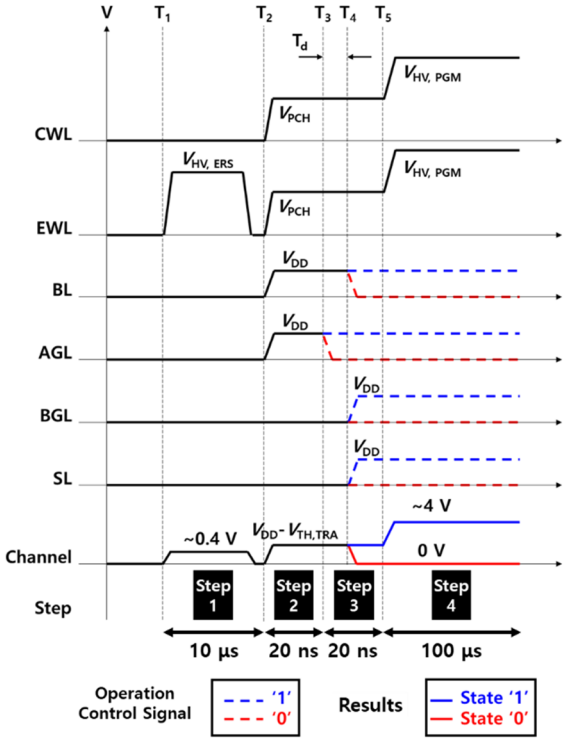



Fig. 7. Timing diagram for Boolean logic operation in NOR eFlash memory array.

Moreover, to store the final results in flash memory, additional NAND flash memory—along with peripheral circuits such as sensing and logic circuitry—would be necessary. In contrast, the proposed technique enables the realization of all 16 Boolean logic functions entirely within logic transistors. It should be noted, however, that the memory used in this work is eFlash memory, which has a larger unit cell size compared to commercial NAND flash memory; in our design, each cell occupies 82 $F^2$.

In a conventional von Neumann architecture, computation is carried out through three distinct steps: 1) reading input data from memory; 2) performing logic operations in the processing unit (ALU); and 3) writing the resulting data back to memory. This architecture inherently requires frequent data transfers between the memory and processing units, leading to inefficiencies in terms of both time and energy. In contrast, the proposed technique reduces the computation cycle to two steps: 1) reading the input data from memory to perform input conversion for the subsequent operation and 2) simultaneously performing logic operations and storing the results directly within the memory array. This integration significantly reduces data movement and improves computational efficiency. Although the proposed system adheres to the stateful logic paradigm—wherein logic states are stored and manipulated directly within memory—it is important to note that as logical input and output are physically different and logic output cannot be directly utilized as logic input for the next step. As a result, the first step-reading of the input data from memory-for intermediate input conversion is always required.

Fig. 7 presents the timing diagram for voltage application corresponding to each step. When the result is "1," the channel

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:34:54 UTC from IEEE Xplore. Restrictions apply.

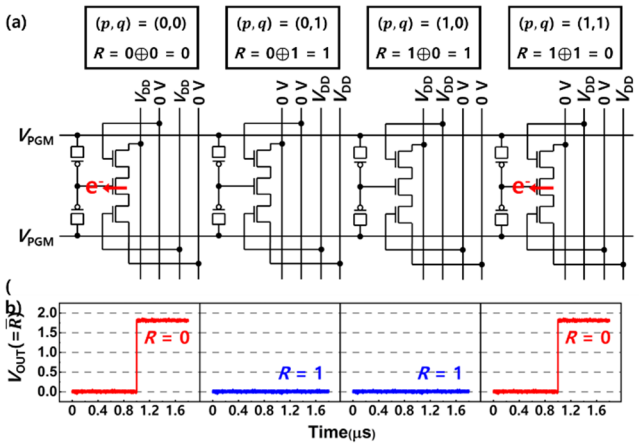



Fig. 8. (a) XOR operation in NOR eFlash memory array and (b) its measurement results. From 0 to 1 $\mu$s, the BL is first precharged and then allowed to develop its voltage level for sensing. At 1 $\mu$s, the sensing latch is enabled, determining the final $V_{OUT}$.

voltage of M3 is boosted, preventing tunneling. Conversely, when the result is "0," the channel voltage of M3 drops to 0 V, allowing tunneling to occur. For cells undergoing the program inhibition operation, it is advisable to apply the control signal to AGL slightly earlier than the other voltages, with a delay interval of Td in *Step 3*, to ensure that the precharged channel potential is maintained.

The proposed array architecture supports row-level parallel operation, as all metal lines carrying the operation control signals are arranged perpendicular to the row (CWL and EWL). This configuration allows for independent application of operation control signals to each cell. Consequently, when the program voltage is applied to a specific row, all cells within that row can perform parallel stateful logic operations. Fig. 8 shows the measurement results of the XOR operation performed on the fabricated chip. After simultaneously performing four XOR operations on four cells in the same row, the results were verified through the read operation. In the case of $R = 0$, the read current does not flow due to the high threshold voltage, causing the output node voltage ($V_{OUT}$) of the sensing circuit to become $V_{DD}$. In the case of $R = 1$, the read current flows due to the low threshold voltage, resulting in $V_{OUT}$ being 0 V.

To demonstrate the applicability of the proposed logic system, a 1-bit full adder is implemented using a sequence of Boolean logic operations within the memory array, as illustrated in Fig. 9. The sum output ($A \oplus B \oplus C_{in}$) is computed via two consecutive XOR operations, where each intermediate result is simultaneously stored in memory upon computation. In order to reuse the intermediate result as the input for a subsequent operation, a read operation is performed, followed by an appropriate voltage setting step to enable the next logic function. The carry-out ($C_{out} = AB + BC + CA$) is derived through sequential AND and OR operations, with all partial results likewise stored in memory. This demonstration confirms that multistep combinational logic functions can be realized entirely through memory-based operations, thereby validating the practical feasibility of the proposed sequential logic-in-memory architecture based on stateful memory operations.

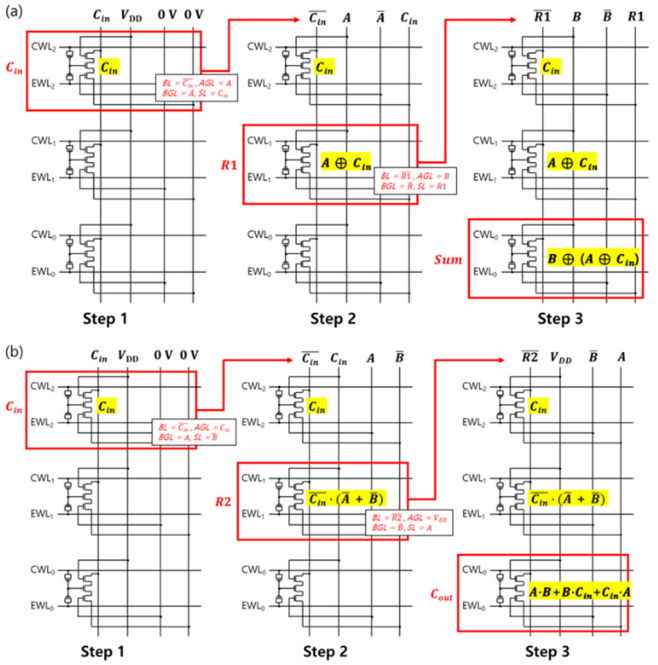



Fig. 9. Method for implementing a 1-bit full adder. (a) Sum computation method. (b) Carry-out computation method.

TABLE II
COMPARISON OF STATEFUL LOGIC DEVICES

| | This Work | [24] | [23] | [20] |
|---|---|---|---|---|
| Device Type | eFlash (3M+2T) | CTF (1T) | FeFET (1T) | RRAM (2R) |
| Logic CMOS Compatibility | Very Good | Good | Good | Poor |
| Program Pulse | 8 V, 100 μs | (Gate/Drain) 5/4 V, 20 μs | 3 V, 0.5 μs | 2 V, 60 ns |
| Read Pulse | 1.8 V, 1 μs | 1 V, 20 μs | 50 mV, 40 ns | 2 V, 60 ns |
| Boolean Operation Energy | 26.76 fJ | 4 fJ | 0.04 fJ | 23.4 fJ |
| Fully CMOS compatible? | Yes | No | No | No |
| Parallel Operation with Array Level? | Yes | No | No | Yes |

If the proposed scheme is extended to multibit input operations, fundamental two-input primitives must be cascaded, requiring additional cells for intermediate states and sequential execution steps. While this would increase the latency for a single logical sequence, the proposed architecture leverages massive array-level parallelism. By executing operations across thousands of rows simultaneously, the system-level throughput remains high, effectively amortizing the latency overhead of individual multistep operations.

Table II summarizes the benchmark analysis of stateful logic operation conditions compared to previous works. While a conservative 100-$\mu$s pulsewidth was used for stability, Fig. 3(c) confirms that a pulsewidth of tens of $\mu$s is sufficient to induce the required threshold voltage shift, implying significant potential for latency reduction. Additionally, transitioning from the current 180-nm legacy node to advanced CMOS

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:34:54 UTC from IEEE Xplore. Restrictions apply.

nodes offers further improvements in switching speed and energy efficiency. Beyond these device-level optimizations, the architecture achieves high-energy efficiency by eliminating the data movement bottleneck of von Neumann systems. Most importantly, the proposed eFlash technology is distinct in its ability to be fully implemented using standard CMOS logic processes. Additionally, for the first time, we demonstrated array-level parallel operation through integration with the driving circuitry.

## IV. Conclusion

In this study, we present, for the first time, an eFlash memory-based sequential logic-in-memory technique tailored for in-memory computing applications, where logic operations are executed and the computation result is stored as a nonvolatile state. This technique facilitates the implementation of 16 arbitrary Boolean logic functions. Based on the NOR array, parallel stateful logic operations at the row level are possible. The proposed method effectively integrates nonvolatile memory devices with CMOS driving circuits, minimizing physical separation between them. A test system, incorporating the eFlash NOR array and driving circuits, was fabricated using a 180-nm low-power CMOS process, and the successful verification of the stateful logic operations was achieved. This work represents the first demonstration of stateful logic using only a standard CMOS logic process, positioning the proposed technology as a highly promising solution for practical in-memory computing applications.

## Acknowledgment

The chip fabrication and electronic design automation (EDA) tool were supported by the IC Design Education Center (IDEC), South Korea.

## References

[1] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, May 2024.

[2] E. Lehtonen and M. Laiho, "Stateful implication logic with memristors," in *Proc. IEEE/ACM Int. Symp. Nanosc. Architectures*, Jul. 2009, pp. 33–36, doi: 10.1109/NANOARCH.2009.5226356.

[3] P. J. Kuekes, D. R. Stewart, and R. S. Williams, "The crossbar latch: Logic value storage, restoration, and inversion in crossbar circuits," *J. Appl. Phys.*, vol. 97, no. 3, Feb. 2005, Art. no. 034301.

[4] H.-S.-P. Wong and S. Salahuddin, "Memory leads the way to better computing," *Nature Nanotechnol.*, vol. 10, no. 3, pp. 191–194, Mar. 2015, doi: 10.1038/nnano.2015.29.

[5] Z.-R. Wang et al., "Functionally complete Boolean logic in 1T1R resistive random access memory," *IEEE Electron Device Lett.*, vol. 38, no. 2, pp. 179–182, Feb. 2017, doi: 10.1109/LED.2016.2645946.

[6] H. Padberg et al., "Experimental demonstration of non-stateful in-memory logic with 1T1R OxRAM valence change mechanism memristors," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 395–399, Jan. 2024.

[7] O. Leitersdorf, R. Ronen, and S. Kvatinsky, "MultPIM: Fast stateful multiplication for processing-in-memory," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1647–1651, Mar. 2022.

[8] J. Reuben et al., "Memristive logic: A framework for evaluation and comparison," in *Proc. PATMOS*, 2017, pp. 1–8.

[9] J. Borghetti, G. S. Snider, P. J. Kuekes, J. J. Yang, D. R. Stewart, and R. S. Williams, "'Memristive' switches enable 'stateful' logic operations via material implication," *Nature*, vol. 464, no. 7290, pp. 873–876, Apr. 2010.

[10] T. You et al., "Exploiting memristive $BiFeO_3$ bilayer structures for compact sequential logics," *Adv. Funct. Mater.*, vol. 24, no. 22, pp. 3357–3365, Jun. 2014, doi: 10.1002/adfm.201303365.

[11] A. Siemon et al., "Realization of Boolean logic functionality using redox-based memristive devices," *Adv. Funct. Mater.*, vol. 25, no. 40, pp. 6414–6423, Oct. 2015, doi: 10.1002/adfm.201500865.

[12] S. Gao, F. Zeng, M. Wang, G. Wang, C. Song, and F. Pan, "Implementation of complete Boolean logic functions in single complementary resistive switch," *Sci. Rep.*, vol. 5, no. 1, p. 15467, Oct. 2015, doi: 10.1038/srep15467.

[13] S. Kvatinsky et al., "MAGIC—Memristor-aided logic," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 61, no. 11, pp. 895–899, Nov. 2014, doi: 10.1109/TCSII.2014.2357292.

[14] P. L. Thangkhiew, R. Gharpinde, and K. Datta, "Efficient mapping of Boolean functions to memristor crossbar using MAGIC NOR gates," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 8, pp. 2466–2476, Aug. 2018, doi: 10.1109/TCSI.2018.2792474.

[15] A. Haj-Ali, R. Ben-Hur, N. Wald, and S. Kvatinsky, "Efficient algorithms for in-memory fixed point multiplication using MAGIC," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2018, pp. 1–5, doi: 10.1109/ISCAS.2018.8351561.

[16] S. Gupta, M. Imani, and T. Rosing, "FELIX: Fast and energy-efficient logic in memory," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2018, pp. 1–7.

[17] M. Cassinerio, N. Ciocchini, and D. Ielmini, "Logic computation in phase change materials by threshold and memory switching," *Adv. Mater.*, vol. 25, no. 41, pp. 5975–5980, Nov. 2013, doi: 10.1002/adma.201301940.

[18] S.-Y. Hu et al., "Reconfigurable Boolean logic in memristive crossbar: The principle and implementation," *IEEE Electron Device Lett.*, vol. 40, no. 2, pp. 200–203, Feb. 2019, doi: 10.1109/LED.2018.2886364.

[19] Z.-R. Wang et al., "Efficient implementation of Boolean and full-adder functions with 1T1R RRAMs for beyond von Neumann in-memory computing," *IEEE Trans. Electron Devices*, vol. 65, no. 10, pp. 4659–4666, Oct. 2018, doi: 10.1109/TED.2018.2866048.

[20] K. Zhang et al., "Rectified tunnel magnetoresistance device with high on/off ratio for in-memory computing," *IEEE Electron Device Lett.*, vol. 41, no. 6, pp. 928–931, Jun. 2020, doi: 10.1109/LED.2020.2987211.

[21] M. Kim, K. Lee, S. Kim, J.-H. Lee, B.-G. Park, and D. Kwon, "Double-gated ferroelectric-gate field-effect-transistor for processing in memory," *IEEE Electron Device Lett.*, vol. 42, no. 11, pp. 1607–1610, Nov. 2021, doi: 10.1109/LED.2021.3116797.

[22] J. Lee, B.-G. Park, and Y. Kim, "Implementation of Boolean logic functions in charge trap flash for in-memory computing," *IEEE Electron Device Lett.*, vol. 40, no. 9, pp. 1358–1361, Sep. 2019, doi: 10.1109/LED.2019.2928335.

[23] M. H. R. Ansari and N. El-Atab, "Efficient implementation of Boolean logic functions using double gate charge-trapping memory for in-memory computing," *IEEE Trans. Electron Devices*, vol. 71, no. 3, pp. 1879–1885, Mar. 2024, doi: 10.1109/TED.2024.3353703.

[24] C. Wu, V. Sreekanti, and J. M. Hellerstein, "Autoscaling tiered cloud storage in anna," *Proc. VLDB Endow.*, vol. 12, no. 6, pp. 624–638, Feb. 2019.

[25] H. Kojima et al., "Embedded flash on 90nm logic technology & beyond for FPGAs," in *IEDM Tech. Dig.*, Dec. 2007, pp. 677–680.

[26] S.-T. Kang et al., "High performance nanocrystal based embedded flash microcontrollers with exceptional endurance and nanocrystal scaling capability," in *Proc. 4th IEEE Int. Memory Workshop*, May 2012, pp. 1–4.

[27] T. Kono et al., "40nm embedded SG-MONOS flash macros for automotive with 160 MHz random access for code and endurance over 10M cycles for data," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2013, pp. 212–213.

[28] S.-H. Song, J. Kim, and C. H. Kim, "A comparative study of single-poly embedded flash memory disturbance, program/erase speed, endurance, and retention characteristic," *IEEE Trans. Electron Devices*, vol. 61, no. 11, pp. 3737–3743, Nov. 2014.

[29] M. Kim, M. Liu, L. R. Everson, and C. H. Kim, "An embedded NAND flash-based compute-in-memory array demonstrated in a standard logic process," *IEEE J. Solid-State Circuits*, vol. 57, no. 2, pp. 625–638, Feb. 2022.

[30] S.-H. Song, K. C. Chun, and C. H. Kim, "A bit-by-bit re-writable eflash in a generic 65 nm logic process for moderate-density nonvolatile memory applications," *IEEE J. Solid-State Circuits*, vol. 49, no. 8, pp. 1861–1871, Aug. 2014.

[31] S.-H. Song, K. C. Chun, and C. H. Kim, "A logic-compatible embedded flash memory for zero-standby power system-on-chips featuring a multi-story high voltage switch and a selective refresh scheme," *IEEE J. Solid-State Circuits*, vol. 48, no. 5, pp. 1302–1314, May 2013.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:34:54 UTC from IEEE Xplore. Restrictions apply.